Sesión 16 — Laboratorio

Programación Declarativa
3º del Grado en Ingeniería Informática
Tipos de datos polimórficos y sus funciones asociadas

Nota

Abre GHCi y carga tus definiciones con `:load fichero.hs` (o `:l fichero.hs`). Recuerda que puedes recargar tras editar con `:reload` (o `:r`). Comprueba los tipos de tus funciones con `:type` (`:t`) antes de probarlas.
Los ejercicios marcados con ★ son los recomendados para completar durante la sesión si el tiempo es limitado.

Parte A — Pila y cola polimórficas

Ejercicio 1. ★

Define `data Pila a = CPila [a]` junto con `push :: a -> Pila a -> Pila a`, `pop :: Pila a -> Maybe (a, Pila a)` y `estaVacia :: Pila a -> Bool`.

```
ghci> pop (push 3 (push 2 (push 1 (CPila []))))
Just (3,CPila [2,1])
ghci> pop (CPila ([] :: [Int]))
Nothing
```

Ejercicio 2.

Define `data Cola a = CCola [a]` representando una cola mediante una única lista cuyo primer elemento es el primero de la cola, junto con `encolar :: a -> Cola a -> Cola a` y `desencolar :: Cola a -> Maybe (a, Cola a)`.

```
ghci> c = encolar 3 (encolar 2 (encolar 1 (CCola [])))
ghci> desencolar c
Just (1,CCola [2,3])
```

Pista: los elementos se encolan al final de la lista y se desencolan por el principio.

Parte B — Árbol general

Ejercicio 3. ★

Define `data ArbolN a = NodoN a [ArbolN a]`, donde cada nodo tiene un valor y una lista (de longitud arbitraria) de hijos. Define `tamanoN :: ArbolN a -> Int` (número total de nodos) y `sumaN :: Num a => ArbolN a -> a` (suma de todos los valores).

```
ghci> arbol = NodoN 1 [NodoN 2 [], NodoN 3 [NodoN 4 []]]
ghci> tamanoN arbol
4
ghci> sumaN arbol
10
```

Ejercicio 4.

Define `profundidadN :: ArbolN a -> Int`, la profundidad máxima del árbol (un nodo sin hijos tiene profundidad 1).

```
ghci> profundidadN arbol
3
```

Ejercicio 5.

Define `mapArbolN :: (a -> b) -> ArbolN a -> ArbolN b`, que aplique una función a todos los valores del árbol conservando su estructura.

```
ghci> mapArbolN (*10) arbol
NodoN 10 [NodoN 20 [],NodoN 30 [NodoN 40 []]]
```

Parte C — El tipo `Either` en varios contextos

Ejercicio 6. ★

Define `dividirSeguro :: Int -> Int -> Either String Int` que divida dos enteros, devolviendo un mensaje de error descriptivo si el divisor es 0.

```
ghci> dividirSeguro 10 2
Right 5
ghci> dividirSeguro 10 0
Left "No se puede dividir entre 0"
```

Ejercicio 7.

Define `validarEdad :: Int -> Either String Int` que valide que una edad sea razonable, devolviendo el motivo del error si no lo es (negativa, o mayor de 120 años).

```
ghci> validarEdad 25
Right 25
ghci> validarEdad (-3)
Left "La edad no puede ser negativa"
ghci> validarEdad 150
Left "Edad fuera de rango razonable"
```

Ejercicio 8.

Define `combinarEither :: Either String a -> Either String b -> Either String (a, b)` que combine dos resultados en un par, solo si **ambos** son `Right`; si alguno es `Left`, el resultado debe ser ese mismo `Left` (el del primero, si ambos fallan).

```
ghci> combinarEither (Right 3) (Right "hola")
Right (3,"hola")
ghci> combinarEither (Left "error A") (Right "hola")
Left "error A"
```

Parte D — Un tipo con tres parámetros: `Triple`

Ejercicio 9.

Define `data Triple a b c = CTriple a b c`, sus tres funciones de proyección (`primero`, `segundo`, `tercero`) y `mapTriple :: (a -> a') -> (b -> b') -> (c -> c') -> Triple a b c -> Triple a' b' c'`, que aplique una función distinta a cada componente.

```
ghci> mapTriple (+1) not length (CTriple 3 True "hola")
CTriple 4 False 4
```

Ejercicio 10.

Define `zipTriple :: [a] -> [b] -> [c] -> [Triple a b c]`, una generalización de `zip` a tres listas a la vez (se detiene en cuanto se agota la más corta).

```
ghci> zipTriple [1,2,3] "ab" [True,False,True]
[CTriple 1 'a' True,CTriple 2 'b' False]
```

Parte E — Árbol binario de búsqueda y conjuntos

Ejercicio 11. ★

Define `data ArbolBin a = HojaBin | NodoBin (ArbolBin a) a (ArbolBin a)` junto con `insertarBT :: Ord a => a -> ArbolBin a -> ArbolBin a` y `perteneceBT :: Ord a => a -> ArbolBin a -> Bool`, manteniendo la propiedad de árbol binario de búsqueda (los valores del subárbol izquierdo de cada nodo son menores, los del derecho son mayores).

```
ghci> arbol = insertarBT 5 (insertarBT 3 (insertarBT 8 HojaBin))
ghci> perteneceBT 3 arbol
True
ghci> perteneceBT 9 arbol
False
```

Ejercicio 12. ★

Define `listaAArbol :: Ord a => [a] -> ArbolBin a` que construya un árbol binario de búsqueda insertando los elementos de una lista uno a uno, en el orden en que aparecen en la lista, usando `insertarBT`.

```
ghci> listaAArbol [5,3,7,1,9]
NodoBin (NodoBin (NodoBin HojaBin 1 HojaBin) 3 HojaBin) 5 (NodoBin
   HojaBin 7 (NodoBin HojaBin 9 HojaBin))
ghci> perteneceBT 7 (listaAArbol [5,3,7,1,9])
True
```

Parte F — Ejercicio integrador: expresiones polimórficas

Ejercicio 13. ★

Define un tipo de expresiones polimórfico:

```
data Expresion a = Lit a
                 | Suma (Expresion a) (Expresion a)
                 | SiMayor (Expresion a) (Expresion a)
                           (Expresion a) (Expresion a)
```

donde `SiMayor e1 e2 e3 e4` evalúa `e3` si el valor de `e1` es mayor que el de `e2`, y `e4` en caso contrario. Define `evaluarExpresion :: (Ord a, Num a) => Expresion a -> a`.

```
ghci> ejemplo = SiMayor (Lit 5) (Lit 3) (Suma (Lit 1) (Lit 1)) (Lit 0)
ghci> evaluarExpresion ejemplo
2
```